React useEffect Hooks

React's **useEffect** hook is a powerful tool that allows developers to interact with the outside world without affecting the rendering or performance of the component it's in. It allows you to synchronize a component with external systems. In other words, the useEffect Hook allows you to perform side effects in your components.

Side effects and examples

Some side effects include fetching data, reading from local storage, registering and deregistering event listeners, directly updating the DOM, and handling timers.

Syntax

```
useEffect(<function>, <dependency>)
```

The **useEffect** hook is implemented with the above syntax. It accepts two arguments, where the second argument (dependency) is optional. This structure allows developers to control when the effect runs and under what conditions.

How does the useEffect Hook get executed?

The following steps are carried out when executing an effect:

1. **Initial render/mounting:** When a functional component containing the **useEffect** hook is initially rendered, the code inside the **useEffect** block runs after the component’s first render. This is similar to the componentDidMount lifecycle method in class components.

2. **Subsequent renders:** You can pass a dependency array as the second argument to the **useEffect** hook. This array contains variables or values that the effect depends on. Any change in these variables will re-render the component. If no dependency array is given, the effect will run on every render.

3. **Cleanup function:** You can also run an optional cleanup function inside the effect. It is used to clean up any resources or subscriptions created by the effect when the component is unmounted or when the dependencies change.

4. **Unmounting:** If a cleanup function is present, it is executed before the component is unmounted from the DOM. This ensures that any resources or subscriptions are properly handled before the component is removed.

Best practice

The best and optimal use of **useEffect** involves including only the minimal set of values that the effect depends on in the dependency array. This practice ensures that the effect runs only when the specified dependencies change, resulting in efficient component re-renders.

© IBM Corporation. All rights reserved.